Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «*Тестирование программного обеспечения*»

Отчёт

По лабораторной работе №1

Вариант: 7001

Студент:

Алферов Г. А.

группа *P3307*

Преподаватель:

Бострикова Д. К.

г. Санкт-Петербург, 2026 г.

Описание задания

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Вариант **7001**

1. Функция arcsin(x)

2. Программный модуль для сортировки массива методом блочной сортировки (http://www.cs.usfca.edu/~galles/visualization/BucketSort.html)

3. Описание предметной области:

Зафод быстро шел по тоннелю. Он сильно нервничал, но пытался скрыть это за целеустремленностью. Он светил фонарем по сторонам. Стены были холодные на ощупь, выложенные темной плиткой, в воздухе стоял запах тления.

Выполнение

Исходный код

https://github.com/wrakelft/SoftwareTestingItmo/tree/main/lab1

Задание 1: *arcsin(x)*

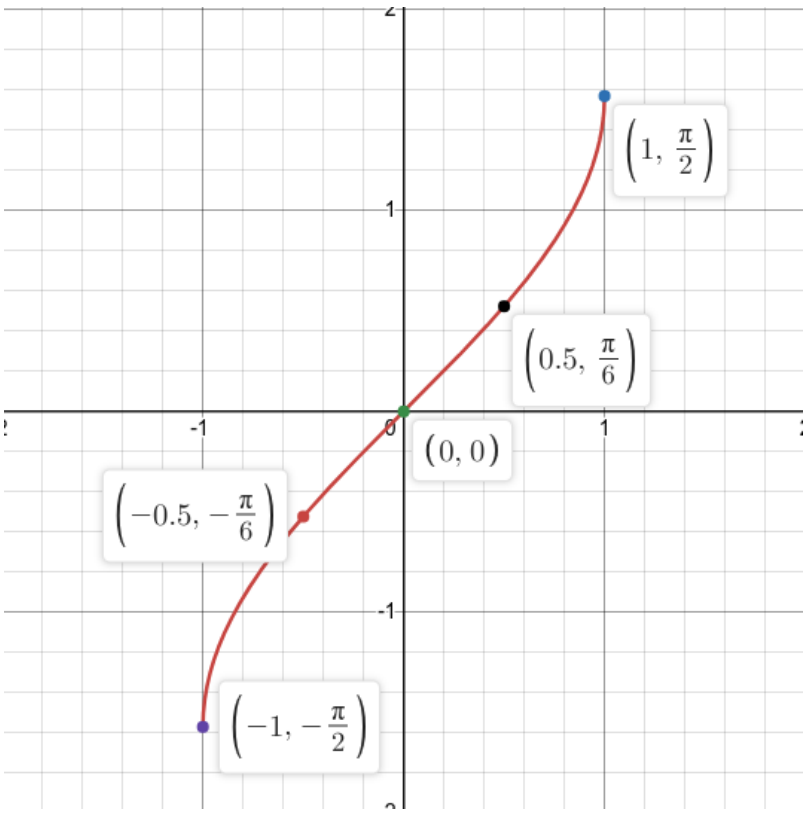


График функции arcsin(x) с табличными значениями в точках.

1. Выберем “интересные” для тестирования точки:
   - **-1.0, 1.0** – граничные значения, вне которых функция не существует, в них значение должно верно определяться с достаточной точностью;
   - **-999, 999** – значения далеко от граничных, функция должна возвращать IllegalArgumentException;
   - **-1.0000001, 1.0000001** – значения близкие к граничным, должны быть верно определены как не входящие в область определения.
   - **-0.99, 0.99** – близкие к граничным точки, должны верно определяться;
   - **-0.5, 0.5** – стандартные табличные значения далеко от угловых случаев;
   - **-0.000001, 0.000001** – близкие к **0** точки, должны считаться верные значения, а не 0;
   - **-0.0** – должно определяться как **-0** ровно, т.е. знак сохраняется в эталонной реализации;
   - **0.0** – должно определяться как **0** ровно, как в эталонной реализации;

- **Double.NaN** – должно определяться как **IllegalArgumentException**;
- **Double.POSITIVE_INFINITY** и **Double.NEGATIVE_INFINITY** - должно определяться как **IllegalArgumentException**;

Таблица тестовых данных для “интересных” точек:

x	arcsin(x)
-1.0	-p/2
-0.99	-1.5608
-0.5	-p/2
-0.000001	-0.000001
0.0	0.0
0.000001	0.000001
0.5	0.5236
0.99	1.5608
1.0	π/2
-999	**IllegalArgumentException**
999	**IllegalArgumentException**
-1.0000001	**IllegalArgumentException**
1.0000001	**IllegalArgumentException**
Double.NaN	**IllegalArgumentException**
Double.POSITIVE_INFINITY	**IllegalArgumentException**
Double.NEGATIVE_INFINITY	**IllegalArgumentException**

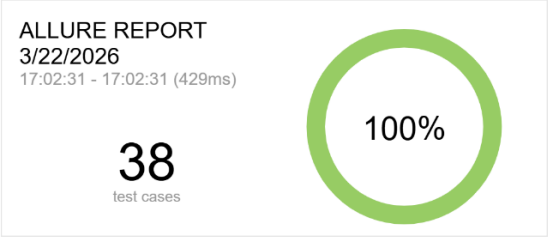
2. Кроме этих точек, будет также проходить тестирование значений в промежутке [-1, 1] с разрывом 0.1, т.е. (**-1.0**, **-0.9**, … **0.9**, **1.0**) и значения

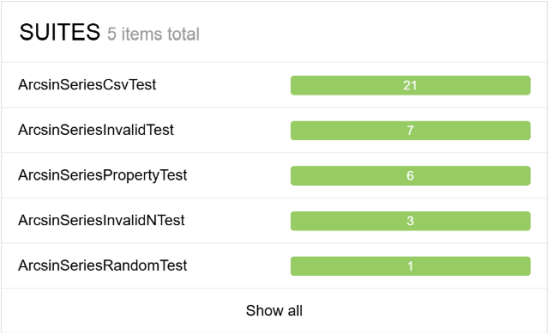
нашей функцией будут сравниваться с эталонной реализацией, чтобы проверять правильность поведения функции на всем протяжении.

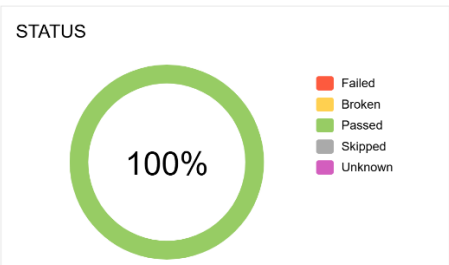
3. Также будет проводиться fuzzy тестирование. В промежутке для тестирования [**-0.99**, **0.99**].

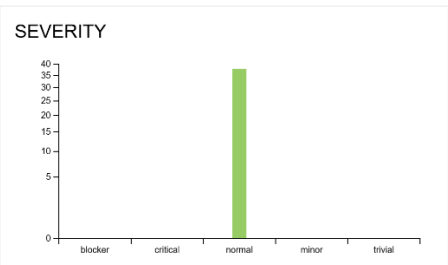
4. Для тестирования функции arcsin(x) использовался преимущественно метод чёрного ящика: были выделены классы эквивалентности, граничные значения и специальные случаи, а результаты сравнивались с эталонной реализацией Math.asin().

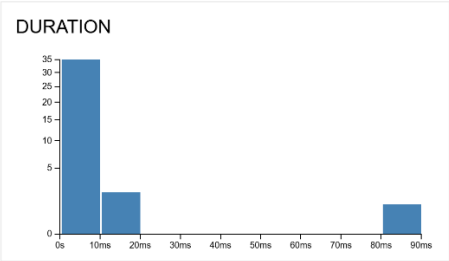
5. Дополнительно использовались элементы белого ящика для проверки внутренних ветвей реализации, таких как обработка -0.0, 0.0 и влияния числа членов ряда n на точность.

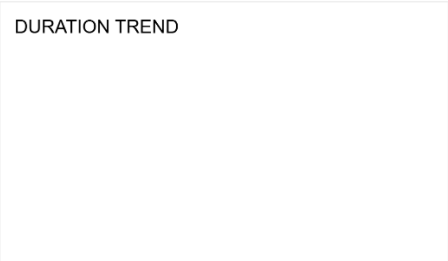












task1 Sessions

task1

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ru.itmo.utils		100 %		100 %	0	13	0	25	0	2	0	1
Total	0 of 121	100 %	0 of 22	100 %	0	13	0	25	0	2	0	1

Created with JaCoCo 0.8.12.202403310830

Тестовое покрытие

Функция **arcsin(x)** корректно обрабатывает все тестовые случаи, включая граничные и специальные значения.

Функция правильно возвращает **IllegalArgumentException** для значений, выходящих за пределы допустимого диапазона, и сохраняет знак для нулевых значений.

Задание 2: *Блочная сортировка*

Эталонная реализация по варианту:
http://www.cs.usfca.edu/~galles/visualization/BucketSort.html

Блочная сортировка – это алгоритм сортировки, при котором элементы сначала распределяются по нескольким ведрам, в зависимости от их значения, затем каждое ведро сортируется отдельно, после чего их содержимое складывается в итоговый массив.

Цель тестирования - убедиться, что реализация алгоритма:

1. Корректно сортирует массивы целых чисел
2. Правильно обрабатывает граничные случаи
3. Корректно проходит все основные этапы алгоритма
4. Соответствует эталонной логике

Для тестирования реализации bucket sort использовалась комбинация подходов:

1. Белый ящик / серый ящик

Так как по условию задания необходимо выделить характерные точки внутри алгоритма и анализировать маршруты прохождения по ним, тестирование проводилось с использованием метода белого ящика (с элементами серого ящика).

Были выделены следующие характерные точки алгоритма:

- **T1** - вход в метод sort
- **T2** - проверка digits == null
- **T3** - проверка digits.length == 0
- **T4** - создание массива ведер
- **T5** - вычисление минимального и максимального значения
- **T6** - проверка случая maxD == minD
- **T7** - распределение элементов по ведрам
- **T8** - вычисление индекса ведра для элемента
- **T9** - сортировка каждого ведра
- **T10** - объединение содержимого ведер в итоговый массив

Для каждого выбранного набора входных данных фиксировался маршрут прохождения по этим точкам.

2. Покрытие кода (Code Coverage)

Проверялось, что:

- все строки кода выполнены хотя бы один раз;
- все ветви условий покрыты тестами;
- протестированы как обычные, так и граничные сценарии работы алгоритма.

Особое внимание уделялось:

- проверке null;
- проверке пустого массива;
- сценарию, при котором все элементы равны;
- распределению элементов по ведрам;
- обработке массивов с повторяющимися значениями.

3. Анализ граничных значений

Проверялись следующие граничные случаи:

- null вместо массива;
- пустой массив;
- массив из одного элемента;
- массив из одинаковых элементов;
- массив с повторяющимися значениями.

4. Сценарное тестирование

Проверялись типичные сценарии использования алгоритма:

- сортировка обычного неупорядоченного массива;
- сортировка уже упорядоченного массива;
- сортировка массива, упорядоченного по убыванию;
- сортировка массива, в котором элементы попадают в разные ведра;

- сортировка массива, в котором часть ведер остается пустой;
- сортировка массива, где несколько элементов попадают в одно ведро.

Выбранные тестовые наборы данных

Для проверки алгоритма использовались следующие входные данные:

- null - проверка обработки недопустимого ввода;
- {} - проверка пустого массива;
- {42} - массив из одного элемента;
- {7, 7, 7, 7} - массив из одинаковых элементов;
- {5, 1, 9, 3, 7} - обычный неупорядоченный массив;
- {1, 2, 3, 4, 5} - уже отсортированный массив;
- {9, 7, 5, 3, 1} - массив, отсортированный по убыванию;
- {5, 1, 5, 3, 1, 7} - массив с повторяющимися значениями;
- {1, 100, 200, 300, 400} - элементы распределяются по разным ведрам;
- {10, 11, 12, 1000} - часть ведер остается пустой;
- {0, 999, 500, 250, 750} - элементы на границах диапазона;
- {21, 22, 23, 24, 25} - несколько элементов попадают в одно ведро.

Маршруты по характерным точкам

Для отдельных тестовых наборов были определены следующие эталонные маршруты:

1. null

Маршрут:
T1 → T2

Ожидаемый результат:
выбрасывается IllegalArgumentException.

2. Пустой массив {}

Маршрут:
T1 → T2(false) → T3

Ожидаемый результат:
возвращается пустой массив.

3. Массив из одинаковых элементов {7, 7, 7, 7}

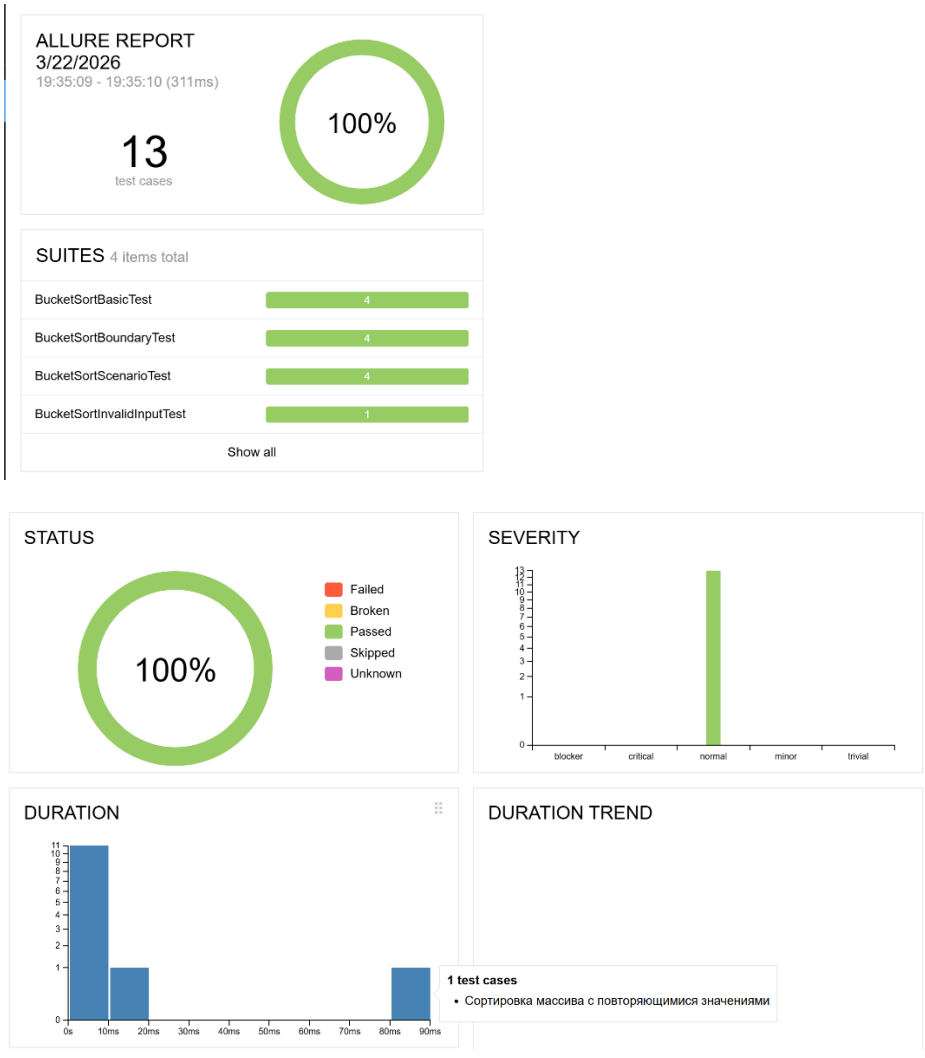
Маршрут:
T1 → T2(false) → T3(false) → T4 → T5 → T6

Ожидаемый результат:
возвращается копия исходного массива.

4. Обычный массив {5, 1, 9, 3, 7}

Маршрут:
T1 → T2(false) → T3(false) → T4 → T5 → T6(false) → T7 → T8 → T9 → T10

Ожидаемый результат:
возвращается отсортированный массив {1, 3, 5, 7, 9}.



task2 Sessions

task2

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ru.itmo.utils		97 %		100 %	1	9	1	24	1	2	0	1
Total	3 of 112	97 %	0 of 14	100 %	1	9	1	24	1	2	0	1

Created with JaCoCo 0.8.12.202403310830

Тестовое покрытие

Результаты тестирования:

1. Корректно сортирует массивы различной структуры
2. Корректно обрабатывает граничные случаи
3. Корректно распределяет элементы по ведрам
4. Правильно сортирует элементы внутри ведер
5. Корректно формирует итоговый массив

Задание 3: *Описание предметной области*

UML диаграмма предметной области.

Предметная область включает в себя:

- Персонажа - Зафод.
- Объекты окружения (тунель, стены, воздух).
- Предметы – фонарь

- Состояния - эмоциональное, внешне демонстрируемое, температура, запах, направление
- Действия (движение, маскировка эмоций, освещение пространства, прикосновение к стенам, восприятие запаха воздуха).

Для тестирования доменной модели использовалась комбинация методов **чёрного ящика** и **белого ящика**.

Чёрный ящик

Тестирование методом чёрного ящика использовалось для проверки поведения сущностей с точки зрения их публичного интерфейса.

При этом проверялись:

- корректность создания объектов;
- корректность изменения состояния после вызова публичных методов;
- правильность возвращаемых значений;
- корректная обработка недопустимых входных данных.

Например:

- проверка изменения эмоционального состояния персонажа через публичный метод setEmotionalState(...);
- проверка того, что фонарь корректно включается, выключается и меняет направление света;

Белый ящик

Элементы метода белого ящика использовались для покрытия внутренних ветвей реализации и проверки специфических участков кода.

Например, отдельно проверялись:

- ветви валидации аргументов конструктора;
- ветви методов, содержащих логические условия;
- изменение внутреннего состояния объектов после выполнения действий;
- случаи автоматического изменения состояния, например автоматическое выключение фонаря при полном разряде батареи.

Цель тестирования - убедиться, что:

1. доменные сущности создаются корректно;
2. состояния объектов и персонажа изменяются в соответствии с логикой предметной области;
3. взаимодействия между объектами модели работают корректно;

Класс SceneObject

- shouldStoreName - проверяет, что базовый объект сцены корректно хранит имя.

Класс Air

- shouldCreateAir - проверяет корректность создания объекта воздуха;

- shouldDetectDecaySmell - проверяет корректное определение запаха тления;
- shouldNotDetectDecaySmellForNormalAir - проверяет, что обычный запах не определяется как запах тления;
- shouldThrowForNegativeIntensity - проверяет обработку недопустимой интенсивности запаха.

Класс Wall

- shouldCreateWall - проверяет корректность создания стены;
- shouldDetectColdWall - проверяет определение холодной стены;
- shouldNotDetectWarmWallAsCold - проверяет, что тёплая стена не считается холодной;
- shouldBecomeTouchedAfterFeelingTemperature - проверяет, что при прикосновении к стене изменяется её внутреннее состояние.

Класс Flashlight

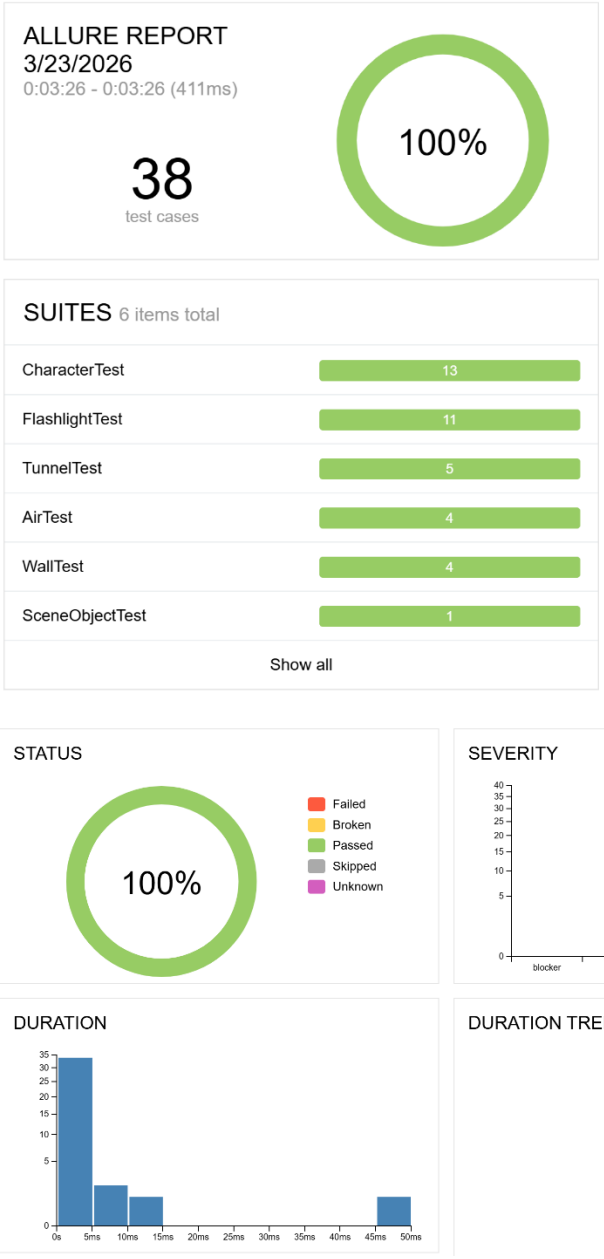
- shouldCreateFlashlight - проверяет корректность создания фонаря;
- shouldTurnOn - проверяет включение фонаря;
- shouldTurnOffManually - проверяет ручное выключение фонаря;
- shouldNotIlluminateWhenTurnedOff - проверяет, что выключенный фонарь не может освещать;
- shouldIlluminateWhenTurnedOn - проверяет, что включённый фонарь с положительным зарядом может освещать;
- shouldShineToDirectionAndConsumeBattery - проверяет изменение направления луча и расход батареи;
- shouldThrowWhenShiningWhileOff - проверяет корректную обработку попытки светить выключенным фонарём;
- shouldThrowWhenBatteryLevelIsNegative - проверяет недопустимое значение заряда батареи меньше нуля;
- shouldThrowWhenBatteryLevelIsGreaterThanHundred - проверяет недопустимое значение заряда батареи больше 100;
- shouldTurnOffAutomaticallyWhenBatteryBecomesZero - проверяет автоматическое выключение фонаря при полном разряде;
- shouldNotIlluminateWithZeroBattery - проверяет, что фонарь не может освещать при нулевом заряде.

Класс Tunnel

- shouldCreateTunnel - проверяет корректность создания тоннеля;
- shouldDetectColdWalls - проверяет, что тоннель определяет наличие холодных стен;
- shouldNotDetectColdWalls - проверяет, что тоннель не считает стены холодными, если обе стены тёплые;
- shouldDetectColdWallsAnotherSide - Определяет наличие холодных стен с другой стороны

Класс Character

- shouldCreateCharacter - проверяет корректность создания персонажа;
- shouldMoveForward - проверяет движение вперёд и увеличение числа шагов;
- shouldMaskEmotion - проверяет изменение отображаемого состояния персонажа;
- shouldNervousState - проверяет что персонаж нервничает;
- shouldChangeEmotionalState - проверяет изменение внутреннего эмоционального состояния;
- shouldThrowWhenSettingNullEmotionalState - проверяет обработку null в эмоциональном состоянии;
- shouldReturnTrueWhenCharacterIsNervous - проверяет корректное определение нервного состояния;
- shouldThrowWhenSettingNullEmotionalState - исключение при попытке установить null в эмоциональное состояние;
- shouldReturnFalseWhenCharacterIsNotNervous - проверяет противоположную ветку метода isNervous();
- shouldTouchLeftWall - проверяет взаимодействие с левой стеной;
- shouldTouchRightWall - проверяет взаимодействие с правой стеной;
- shouldSmellAir - проверяет восприятие запаха воздуха;
- shouldShineFlashlight - проверяет использование фонаря персонажем;
- shouldReturnCurrentTunnel - проверяет корректное хранение текущего тоннеля персонажа.



task3

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ru.itmo.model		100 %		96 %	1	53	0	84	0	40	0	6
ru.itmo.enums		100 %		n/a	0	7	0	23	0	7	0	7
Total	0 of 425	100 %	1 of 26	96 %	1	60	0	107	0	47	0	13

Created with JaCoCo 0.8.12.202403310830

Тестовое покрытие

Выводы

Во время выполнения лабораторной работы я научился работать с JUnit5. Составил юнит тесты, разработал тестовое покрытие для собственной доменной модели. Получил опыт в использовании утилит для метрик, таких как Allure и Jacoco.